XENSIV™ TLx4971 - TLE4972 Current Sensor Programmer - Serial Commands Interface

Document Information

Item	Details
Document Title	Serial Commands Interface
Product	XENSIV™ TLx4971 - TLE4972 Current Sensor Programmer
Revision	1.1
Date	March 30, 2026
Status	Released

Revision History

Revision	Date	Author	Description
1.0	July 22, 2026	-	Initial release

Related Documents

- **XENSIV™ TLE4972 user manual**, Rev. 1.0.1 (2022-10-25) - Sensor SICI interface, internal registers, EEPROM content, CRC and calibration methods
- XENSIV™ TLE4972/TLx4971 Current sensor programmer user guide - Hardware setup, connection diagrams, software installation and usage etc.
- XENSIV™ TLx4971 - TLE4972 Current Sensor Programmer app - Evaluation Software
- FTDI Drivers - D2XX drivers for USB serial communication (included with software)

Scope

Purpose

This document provides a comprehensive technical reference for the serial communication protocol used by the XENSIV™ TLx4971 - TLE4972 Current Sensor Programmer firmware. It describes the command structure, data formats, and operational procedures required to communicate with the evaluation kit programmer via USB serial interface.

Target Audience

This documentation is intended for:

- Software developers integrating the evaluation kit into custom applications
- Test engineers developing automated test scripts
- Technical support personnel troubleshooting communication issues
- System integrators working with the TLE480x sensor evaluation platform

Document Coverage

This document covers:

- **Bootloader Mode**: Communication protocol for firmware updates
- **Firmware Mode**: Complete command set for sensor operation, configuration, and data acquisition
- **Protocol Specifications**: Packet formats, checksums, and data encoding
- **Command Reference**: Detailed syntax and examples for all supported commands
- **Asynchronous Data**: Continuous readout packet formats and register mappings
- **Worked Procedures**: EEPROM CRC handling, OCD threshold programming, and the Double Code Word calibration flow expressed as serial command sequences

Out of Scope

This document does NOT cover:

- Hardware design, schematics, or electrical specifications (see Evaluation Board User Guide)
- Magnetic field theory, calibration accuracy analysis, or the derivation of the calibration formulas (see the TLE4972 user manual)
- Graphical User Interface (GUI) software operation
- Mechanical installation or mounting guidelines
- Safety and regulatory compliance information

Prerequisites

Before using this documentation:

- Review the **XENSIV™ TLE4972/TLx4971 Current sensor programmer user guide** for hardware setup
- Ensure proper USB driver installation for COM port enumeration
- Understand basic serial communication concepts (UART, baud rate, checksums)
- Have familiarity with hexadecimal notation and bitwise operations

---

Important: EEPROM CRC Handling

> **WARNING — The CRC is never recalculated automatically.**
>
> Neither the programmer firmware, nor the MCU, nor the sensor recalculates the sensor EEPROM CRC. **The host application is solely responsible** for computing the correct CRC and embedding it in the data stream before issuing any write operation that ends up in the sensor EEPROM.
>
> **An incorrect CRC is not rejected at write time.** It is only detected by the sensor's internal safety mechanism after the next start-up, and the sensor then signals a permanent fault: the **OCD open-drain outputs are driven to GND** and the part is unusable until a valid image is reprogrammed.

Rules

1. **Always read the full EEPROM first.** The CRC covers all 18 words, not only the ones you change. Read the current image with `cmdReadEEPROM` (`0x10`), modify it in place, then recalculate.
2. **Recalculate after every modification**, including single-bit changes.
3. **Write the CRC back into word 2, bits 7:0** (address line `0x42`, field `CRC`) of the image you are about to program. The remaining bits of word 2 must be preserved.
4. **Verify by reading back.** After programming and a sensor power cycle, re-read the EEPROM and confirm both the payload and the CRC.

CRC Specification

Defined in the TLE4972 user manual, chapter 5.2:

Property	Value
Algorithm	CRC-8 (SAE-J1850)
Polynomial	`0x1D` — x^8 + x^4 + x^3 + x^2 + 1
Seed	`0xAA`
Final step	Bitwise inversion (`^ 0xFF`)
Byte order	EEPROM line 3 → 17 (MSB then LSB of each word), then line 0 → 1, then the **high byte of line 2**
Excluded	Low byte of line 2 — this is the CRC field itself
Stored at	Word 2, bits 7:0

Reference Implementation ©

```c
#include <stdbool.h>
#include <stdint.h>

#define EEPROM_LINE_COUNT (18u)
#define CRC_POLYNOMIAL    (0x1Du)   /* x^8 + x^4 + x^3 + x^2 + 1 */
#define CRC_SEED          (0xAAu)

uint8_t eepromCrcCalc(const uint16_t *eeprom)
{
    uint8_t crc = CRC_SEED;

    /* Byte stream: line 3..17, then line 0..1, then the high byte of line 2.
     * The low byte of line 2 holds the CRC itself and is excluded. */
    for (uint32_t i = 0u; i < (EEPROM_LINE_COUNT * 2u) - 1u; i++)
    {
        uint16_t word = eeprom[((i / 2u) + 3u) % EEPROM_LINE_COUNT];
        uint8_t byte = ((i & 1u) == 0u) ? (uint8_t)(word >> 8) : (uint8_t)word;

        crc ^= byte;
        for (uint8_t bit = 0u; bit < 8u; bit++)
        {
            uint32_t shifted = (uint32_t)crc << 1;
            crc = ((crc & 0x80u) != 0u) ? (uint8_t)(shifted ^ CRC_POLYNOMIAL)
                                        : (uint8_t)shifted;
        }
    }

    return (uint8_t)(crc ^ 0xFFu);
}

bool eepromCrcCheck(const uint16_t *eeprom)
{
    return (uint8_t)(eeprom[2] & 0xFFu) == eepromCrcCalc(eeprom);
}
```

Patching the recalculated CRC into the image:

```c
eeprom[2] = (uint16_t)((eeprom[2] & 0xFF00u) | eepromCrcCalc(eeprom));
```

A worked end-to-end sequence is given in Complete Programming Example.

Hardware Prerequisites

Hardware Connection

Before using the serial commands documented here, ensure proper hardware setup:

1. **Physical Connection**: Connect the evaluation kit programmer to your PC via USB
2. **Power Supply**: Verify the evaluation board is properly powered
3. **Sensor Installation**: Ensure the TLE480x sensor is correctly connected to the evaluation board

Important: For detailed hardware setup instructions, connection diagrams, and safety information, please refer to the **XENSIV™ TLE4972/TLx4971 Current sensor programmer user guide**.

COM Port Detection

After connecting the hardware:

1. The programmer will enumerate as a virtual COM port on your system
2. Identify the COM port number from your operating system's device manager
3. Use this COM port for all serial communication as described in this document

---

Bootloader Mode

Overview

Before entering firmware mode, the MCU operates in bootloader mode. The bootloader provides basic communication validation and firmware update capabilities.

Connection Setup

Serial Port Configuration:

- **Baud Rate**: 1,250,000
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None

Bootloader Sequence

Step 1: Power On

Power on the programmer device.

Step 2: Connect to COM Port

Establish serial connection using the configuration above.

Step 3: Bootloader Handshake

Sequence 1 - Echo Command:

- **Send**: `0x09` (9 decimal) - Echo command
- **MCU Response**: `0x20` (32 decimal) - Bootloader acknowledgment

Sequence 2 - Switch to Firmware Mode:

- **Send**: `0x12` (18 decimal) - Exit bootloader and start firmware
- **MCU Response**: `0x20` (32 decimal) - Acknowledgment before restart
- **Result**: MCU performs Alternative Boot Mode 0 (ABM0) restart and enters firmware mode

Bootloader Commands

Command	Decimal	Description	Response
0x09	9	Echo - Validate communication	0x20 (32)
0x10	16	Prepare page write (512 bytes)	0x20 (32)
0x11	17	Write page to flash	0x20 (32)
0x12	18	Finalize and restart to firmware	0x20 (32) then restart

Firmware Update Process

Firmware updates must be performed separately using the **Evaluation Kit Software**. The bootloader supports page-based flash programming:

1. Send echo command (0x09) for validation
2. Send prepare page command (0x10)
3. Transmit 512-byte page content
4. Send write page command (0x11) to commit to flash
5. Repeat steps 2-4 for all pages
6. Send restart command (0x12) to complete update

Note: The bootloader uses Alternative Boot Mode 0 (ABM0) with flash start address `0x08010000` (Physical Sector 1).

Transitioning to Firmware Mode

Once the bootloader sequence is complete and the MCU restarts, all subsequent commands follow the **Firmware Mode Protocol** documented below.

Firmware Mode Protocol

After successfully completing the bootloader sequence, the MCU operates in firmware mode where the following command protocol applies.

Serial Port Configuration

Firmware mode uses the **same serial port settings** as the bootloader:

- **Baud Rate**: 1,250,000
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None
- **Flow Control**: None

Protocol Overview

All host-to-MCU communication in firmware mode uses a fixed 5-byte command header, optionally followed by a data payload. Every command begins with the start identifier byte `0xAA`. Multi-byte sensor register values are transmitted **MSB first** (big-endian), while the packed ADC readout stream uses the byte layout described in its own section.

Command Frame (Host → MCU)

The host always sends a 5-byte header first:

Offset	Field	Size	Description
0	`START`	1	Start identifier, always `0xAA`
1	`CMD`	1	Command code (see command table)
2	`SENSOR_SELECT`	1	Target sensor index: `0` = Sensor 1, `1` = Sensor 2, `2` = Sensor 3
3	`N_DATA`	1	Number of payload data bytes that follow the header (`0` if none)
4	`N_RESP`	1	Number of response data bytes the host expects (informational, echoed back in the acknowledge frame)

If `N_DATA > 0`, the host transmits `N_DATA` additional payload bytes **after** receiving the intermediate acknowledge frame (see handshake below).

> **Note**: If the first byte received is not `0xAA`, the MCU discards it and resynchronizes, waiting for a valid start identifier.

Acknowledge / Response Frame (MCU → Host)

Every acknowledge frame is exactly 2 bytes:

Offset	Field	Size	Description
0	`CMD_ECHO`	1	Echoed command code (see caveat below)
1	`N_RESP`	1	Echoed expected response length (`N_RESP` from the command header)

Any additional response payload (register values, EEPROM contents, temperature, version, etc.) is sent **immediately after** the acknowledge frame, as described per command.

> **CMD_ECHO caveat**: For commands that carry a payload, the firmware overwrites the echoed command byte with the value of payload byte `data[1]` before sending the final acknowledge frame. This is intentional firmware behavior; host software should key off the transaction it initiated rather than relying solely on `CMD_ECHO` for these commands. The commands affected are marked in the command table.

Transaction Handshake

The exact exchange depends on whether the command carries a payload:

Commands without payload (`N_DATA = 0`)

```
Host → MCU:  [0xAA][CMD][SENSOR_SELECT][0x00][N_RESP]
MCU  → Host:  [CMD_ECHO][N_RESP]                    (acknowledge)
MCU  → Host:  [ ...optional response payload... ]    (if any)
```

Commands with payload (`N_DATA > 0`)

```
Host → MCU:  [0xAA][CMD][SENSOR_SELECT][N_DATA][N_RESP]
MCU  → Host:  [CMD][N_RESP]                          (intermediate acknowledge)
Host → MCU:  [ D0 ][ D1 ] ... [ D(N_DATA-1) ]        (payload)
MCU  → Host:  [CMD_ECHO][N_RESP]                     (final acknowledge)
MCU  → Host:  [ ...optional response payload... ]    (if any)
```

> **Important**: Payload-carrying commands produce **two** acknowledge frames — one intermediate handshake (echoing the original `CMD`) sent before the payload is accepted, and one final acknowledge (echoing `data[1]` as `CMD_ECHO`) sent after processing. Payload-free commands produce only a single acknowledge frame.

Firmware Command Reference

Command Summary

Command	Code	Payload	Response Payload	Description
`cmdReadEEPROM`	`0x10`	none	36 bytes	Read sensor OTP/EEPROM (18 × 16-bit words)
`cmdEnterTestMode`	`0x11`	none	none	Enter sensor test/SICI mode and power down ISM
`cmdResetSensor`	`0x13`	none	none	Power-cycle the selected sensor supply (VDD off/on)
`cmdStartReadout`	`0x14`	none	continuous stream	Start continuous ADC readout streaming
`cmdStopReadout`	`0x15`	none	none	Stop ADC readout streaming
`cmdOscFunction`	`0x16`	6 bytes	none	Configure oscilloscope/trigger mode
`cmdReadRegister`	`0x17`	≥ 2 bytes	none	Read a sensor register into internal buffer
`cmdGetRegisterValue`	`0x18`	none	2 bytes	Retrieve the last register value read
`cmdResetToBootloader`	`0x19`	none	none	Restart the MCU into bootloader mode
`cmdFwVersion`	`0x21`	none	2 bytes	Read firmware version
`cmdInitMCU`	`0x22`	≥ 2 bytes	none	Initialize MCU and sensor array configuration
`cmdProgramEXTEEPROM`	`0x23`	17 bytes	none	Write 17 bytes to the external I^2C EEPROM
`cmdReadEXTEEPROM`	`0x24`	≥ 2 bytes	none	Read 16 bytes from external I^2C EEPROM into buffer
`cmdGetExtEEPROMRegVal`	`0x25`	none	16 bytes	Retrieve the last external EEPROM read
`cmdProgramEEPROM`	`0x26`	36 bytes	none	Program sensor OTP/EEPROM (18 × 16-bit words)
`cmdCheckIfExtEEPROMIsPresent`	`0x27`	none	2 bytes	Detect presence of external I^2C EEPROM

Command	Code	Payload	Response Payload	Description
`cmdReadTemperature`	`0x28`	none	6 bytes	Read temperature register of all three sensors
`cmdEnableMux`	`0x29`	2 bytes	none	Enable/disable and select the VREF multiplexer
`cmdSetRegister`	`0x30`	3 bytes	none	Write a value to a sensor register
`cmdCalibrateBoard`	`0x31`	none	3 bytes	Read averaged VSENS calibration ADC value
`cmdWriteFlashSector`	`0x32`	≤ 14 bytes	none	Write a data block to the reserved flash sector
`cmdReadFlashSector`	`0x33`	none	14 bytes	Read the reserved flash sector data block

---

Command Details

`0x10` — Read Sensor EEPROM (`cmdReadEEPROM`)

Reads the full sensor OTP/EEPROM contents of the selected sensor.

- **Send**: `[0xAA][0x10][SENSOR_SELECT][0x00][0x24]`
- **Reply**:
  1. Acknowledge frame `[0x10][0x24]`
  2. **36 data bytes** = 18 consecutive 16-bit words, each transmitted **MSB first**: `[W0_H][W0_L][W1_H][W1_L] ... [W17_H][W17_L]`

Each word corresponds to one EEPROM/OTP address of the sensor.

Example — read the EEPROM of Sensor 1:

```
Host → MCU:  AA 10 00 00 24
MCU  → Host:  10 24                                (acknowledge)
MCU  → Host:  1A 2B 00 04 FF F0 ... (36 bytes total)  (18 words, MSB first)
```

Here word 0 = `0x1A2B`, word 1 = `0x0004`, word 2 = `0xFFF0`, and so on.

`0x11` — Enter Test Mode (`cmdEnterTestMode`)

Puts the selected sensor into SICI test mode (sends the interface password) and powers down the sensor's internal signal-processing module (ISM).

- **Send**: `[0xAA][0x11][SENSOR_SELECT][0x00][0x00]`
- **Reply**: Acknowledge frame `[0x11][0x00]`

Example — put Sensor 2 into test mode:

```
Host → MCU:  AA 11 01 00 00
MCU  → Host:  11 00
```

`0x13` — Reset Sensor (`cmdResetSensor`)

Power-cycles the sensor supply (VDD off for ~100 ms, then on) to reset the sensor.

- **Send**: `[0xAA][0x13][SENSOR_SELECT][0x00][0x00]`
- **Reply**: Acknowledge frame `[0x13][0x00]`

> **Note**: The programmer drives a **single, shared 3.3 V sensor supply rail**. This command therefore power-cycles **all three sensor sockets** regardless of the `SENSOR_SELECT` value, and clears the SICI test mode of every sensor.

> **Required before `cmdStartReadout`**: A sensor left in SICI test mode has its ISM powered down and produces no valid AOUT/VREF signal. Always issue this command before `cmdStartReadout` (`0x14`) to return the sensors to normal operating mode. Allow the supply to settle after the reset before starting acquisition.

Example — reset Sensor 1:

```
Host → MCU:  AA 13 00 00 00
MCU  → Host:  13 00
```

`0x14` — Start Readout (`cmdStartReadout`)

Starts continuous acquisition. After the acknowledge frame, the MCU begins streaming fixed 12-byte ADC frames (see ADC Readout Stream Format) driven by the internal PWM sampling timer until a `cmdStopReadout` is received.

Required Start-Up Sequence

The firmware performs **no** implicit reset or reference configuration. The host must execute the following sequence before every readout:

Step	Command	Purpose
1	`cmdResetSensor` (`0x13`)	Power-cycle the sensor rail. Leaves SICI test mode and re-enables the ISM.
2	*wait ≥ 100 ms*	Supply settling and sensor start-up.
3	`cmdEnableMux` (`0x29`)	Configure the on-board VREF reference — **depends on the sensor output mode**, see below.
4	`cmdStartReadout` (`0x14`)	Begin streaming.

Step 3 — VREF mux setting by output mode

The required mux state is derived from the `OPMODE` field of the sensor EEPROM (word 0, bits 6:5):

`OPMODE`	Output mode	`cmdEnableMux` payload	Meaning
`3`	Single-ended (SE)	`01 01`	Mux **enabled**, reference applied to all sensor VREF lines
`0`, `1`, `2`	SDBID / FD / SDUNI	`00 00`	Mux **disabled**, all VREF mux lines driven low

In single-ended mode the sensor expects an externally supplied reference on its VREF pin, so the on-board mux must provide it. In the differential modes the sensor drives VREF itself and the mux must stay off to avoid contention.

> **Note**: `cmdEnableMux` acts on all three sensor VREF lines at once and ignores `SENSOR_SELECT` (the evaluation software always sends `0x00`).

- **Send**: `[0xAA][0x14][SENSOR_SELECT][0x00][0x00]`
- **Reply**:
  1. Acknowledge frame `[0x14][0x00]`
  2. Continuous 12-byte ADC readout frames

Example — full start-up for a sensor in fully-differential mode (Sensor 1):

```
Host → MCU:  AA 13 00 00 00                       (1. cmdResetSensor)
MCU  → Host:  13 00

             ... wait 100 ms ...                  (2. settling)

Host → MCU:  AA 29 00 02 00                       (3. cmdEnableMux)
MCU  → Host:  29 00                                (intermediate ack)
Host → MCU:  00 00                                 (payload: disable, level 0 — not single-ended)
MCU  → Host:  00 00                                (final ack, CMD_ECHO = data[1] = 0x00)

Host → MCU:  AA 14 00 00 00                       (4. cmdStartReadout)
MCU  → Host:  14 00                                (acknowledge)
MCU  → Host:  AA 34 12 00 ... 7B                   (12-byte frame, repeats)
MCU  → Host:  AA 35 12 00 ... 7C                   (next 12-byte frame)
                     ...                           (until cmdStopReadout)
```

For a sensor configured in **single-ended** mode, step 3 becomes:

```
Host → MCU:  AA 29 00 02 00
MCU  → Host:  29 00                                (intermediate ack)
Host → MCU:  01 01                                 (payload: enable, level select 1)
MCU  → Host:  01 00                                (final ack, CMD_ECHO = data[1] = 0x01)
```

See ADC Readout Stream Format for decoding a frame.

`0x15` — Stop Readout (`cmdStopReadout`)

Stops the ADC readout streaming and halts the sampling PWM timers.

- **Send**: `[0xAA][0x15][SENSOR_SELECT][0x00][0x00]`
- **Reply**: Acknowledge frame `[0x15][0x00]`

Example — stop streaming:

```
Host → MCU:  AA 15 00 00 00
MCU  → Host:  15 00
```

`0x16` — Oscilloscope / Trigger Configuration (`cmdOscFunction`)

Configures triggered (“oscilloscope”) acquisition parameters.

- **Send**: `[0xAA][0x16][SENSOR_SELECT][0x06][N_RESP]` followed by 6 payload bytes:

Payload byte	Field	Description
`data[0]`	Osc mode	`0` = disabled, `1` = enabled
`data[1]`	Threshold LSB	Low byte of the 16-bit voltage trigger threshold
`data[2]`	Threshold MSB	High byte of the 16-bit voltage trigger threshold (`threshold = (data[2] << 8) + data[1]`)
`data[3]`	Sensor select	Sensor used for triggering (`1` , `2` , or `3`)
`data[4]`	Edge	Trigger edge: `1` = rising, `2` = falling
`data[5]`	PWM/speed	`1` = high speed / 500 ms window, `2` = normal / 500 ms window, `3` = normal / 1 s window

- **Reply**: Intermediate acknowledge `[0x16][N_RESP]` , then final acknowledge `[data[1]][N_RESP]`

Example — enable osc mode, threshold `0x0800` (2048), trigger on Sensor 1, rising edge, normal speed:

```
Host → MCU:  AA 16 00 06 00
MCU  → Host:  16 00                                 (intermediate ack)
Host → MCU:  01 00 08 01 01 02                      (payload: on, TH_L, TH_H, sensor, edge, speed)
MCU  → Host:  00 00                                 (final ack, CMD_ECHO = data[1] = 0x00)
```

`0x17` — Read Register (`cmdReadRegister`)

Reads a single sensor register over SICI and stores the result internally. Retrieve the value with `cmdGetRegisterValue` (`0x18`).

- **Send**: `[0xAA][0x17][SENSOR_SELECT][0x02][N_RESP]` followed by 2 payload bytes:

Payload byte	Field	Description
`data[0]`	(reserved)	Not used by the read operation
`data[1]`	Register address	Address of the sensor register to read

- **Reply**: Intermediate acknowledge `[0x17][N_RESP]` , then final acknowledge `[data[1]][N_RESP]`

> The register value itself is **not** returned by this command. Issue `cmdGetRegisterValue` (`0x18`) afterwards to read it.

Example — read register `0x18` (24) of Sensor 1:

```
Host → MCU:  AA 17 00 02 00
MCU  → Host:  17 00                                 (intermediate ack)
Host → MCU:  00 18                                  (payload: reserved, register address)
MCU  → Host:  18 00                                 (final ack, CMD_ECHO = data[1] = 0x18)
```

Follow with `cmdGetRegisterValue` (`0x18`) to fetch the value.

`0x18` — Get Register Value (`cmdGetRegisterValue`)

Returns the value read by the most recent `cmdReadRegister` .

- **Send**: `[0xAA][0x18][SENSOR_SELECT][0x00][0x02]`
- **Reply**:
  1. Acknowledge frame `[0x18][0x02]`
  2. **2 data bytes** = 16-bit register value, **MSB first**: `[VAL_H][VAL_L]`

Example — fetch the value read by the previous `cmdReadRegister` (value `0x0ABC`):

```
Host → MCU:  AA 18 00 00 02
MCU  → Host:  18 02                              (acknowledge)
MCU  → Host:  0A BC                               (register value = 0x0ABC)
```

`0x19` — Reset to Bootloader (`cmdResetToBootloader`)

Restarts the MCU back into bootloader mode (for firmware update). After the acknowledge, the MCU waits ~1 ms and performs the bootloader restart.

- **Send**: `[0xAA][0x19][SENSOR_SELECT][0x00][0x00]`
- **Reply**: Acknowledge frame `[0x19][0x00]` , then the device restarts into the bootloader.

Example — jump to the bootloader:

```
Host → MCU:  AA 19 00 00 00
MCU  → Host:  19 00                              (then device restarts into bootloader)
```

`0x21` — Firmware Version (`cmdFwVersion`)

Reads the firmware version number.

- **Send**: `[0xAA][0x21][SENSOR_SELECT][0x00][0x02]`
- **Reply**:
  1. Acknowledge frame `[0x21][0x02]`
  2. **2 data bytes**: `[0x00][VERSION]` where `VERSION` is the firmware version (currently `50` / `0x32`)

Example — read the firmware version:

```
Host → MCU:  AA 21 00 00 02
MCU  → Host:  21 02                                  (acknowledge)
MCU  → Host:  00 32                                   (version = 0x32 = 50)
```

`0x22` — Initialize MCU (`cmdInitMCU`)

Initializes the MCU peripherals and sensor array configuration. Must be issued before sensor operations.

- **Send**: `[0xAA][0x22][SENSOR_SELECT][N_DATA][N_RESP]` followed by payload:

Payload byte	Field	Description
`data[0]`	Array config	Sensor array configuration selector (`0` – `5`, see below)
`data[1]`	(echo)	Value echoed back as `CMD_ECHO`

Array Configuration Selector

The evaluation board carries **three physical sensor sockets/slots** (referred to here as *Slot A*, *Slot B*, and *Slot C* — the fixed hardware positions). The array config byte selects which physical slot is mapped to each **logical sensor index** (Sensor 1 / Sensor 2 / Sensor 3) used by all other commands via the `SENSOR_SELECT` field.

This lets the host software remap the sensor order without physically re-wiring the board — for example, to match the order in which the operator inserted the parts, or to test the six possible orderings. All six permutations of the three slots are supported:

`data[0]`	Logical Sensor 1	Logical Sensor 2	Logical Sensor 3
`0`	Slot A	Slot B	Slot C
`1`	Slot A	Slot C	Slot B
`2`	Slot B	Slot A	Slot C
`3`	Slot B	Slot C	Slot A
`4`	Slot C	Slot A	Slot B
`5`	Slot C	Slot B	Slot A

Notes:

- `data[0] = 0` is the default 1:1 mapping (logical index = physical slot).
- Values outside `0` – `5` leave the previously active mapping unchanged.
- The selected mapping persists until the next `cmdInitMCU` or MCU reset, and it affects every subsequent command that uses `SENSOR_SELECT` (register access, readout, EEPROM, temperature, etc.).
- **Reply**: Intermediate acknowledge `[0x22][N_RESP]`, then final acknowledge `[data[1]][N_RESP]`

Example — initialize the MCU with array configuration `0x01`:

```
Host → MCU:  AA 22 00 02 00
MCU  → Host:  22 00                                  (intermediate ack)
Host → MCU:  01 00                                   (payload: array config, echo byte)
MCU  → Host:  00 00                                  (final ack, CMD_ECHO = data[1] = 0x00)
```

`0x23` — Program External EEPROM (`cmdProgramEXTEEPROM`)

Writes 17 bytes to the on-board external I²C EEPROM. The MCU raises the 3.3 V sensor supply and waits ~5 ms before writing.

- **Send**: `[0xAA][0x23][SENSOR_SELECT][0x11][N_RESP]` followed by **17 payload bytes** (`data[0]` is the EEPROM start address, `data[1..16]` the data to write).
- **Reply**: Intermediate acknowledge `[0x23][N_RESP]`, then final acknowledge `[data[1]][N_RESP]`

Example — write 16 bytes starting at external EEPROM address `0x00`:

```
Host → MCU:  AA 23 00 11 00
MCU  → Host:  23 00                                  (intermediate ack)
Host → MCU:  00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF 10   (17 bytes: addr + 16 data)
MCU  → Host:  11 00                                  (final ack, CMD_ECHO = data[1] = 0x11)
```

`0x24` — Read External EEPROM (`cmdReadEXTEEPROM`)

Reads 16 bytes from the external I²C EEPROM starting at the given address and stores them internally. Retrieve them with `cmdGetExtEEPROMRegVal` (`0x25`).

- **Send**: `[0xAA][0x24][SENSOR_SELECT][0x02][N_RESP]` followed by 2 payload bytes:

Payload byte	Field	Description
`data[0]`	(reserved)	Not used
`data[1]`	Start address	EEPROM byte address to start reading from

- **Reply**: Intermediate acknowledge `[0x24][N_RESP]`, then final acknowledge `[data[1]][N_RESP]`

> The 16 read bytes are retrieved with `cmdGetExtEEPROMRegVal` (`0x25`).

Example — read 16 bytes from external EEPROM starting at address `0x00` :

```
Host → MCU:  AA 24 00 02 00
MCU  → Host:  24 00                                  (intermediate ack)
Host → MCU:  00 00                                   (payload: reserved, start address)
MCU  → Host:  00 00                                  (final ack, CMD_ECHO = data[1] = 0x00)
```

Follow with `cmdGetExtEEPROMRegVal` (`0x25`) to fetch the 16 bytes.

`0x25` — Get External EEPROM Value (`cmdGetExtEEPROMRegVal`)

Returns the 16 bytes read by the most recent `cmdReadEXTEEPROM` .

- **Send**: `[0xAA][0x25][SENSOR_SELECT][0x00][0x10]`
- **Reply**:
  1. Acknowledge frame `[0x25][0x10]`
  2. **16 data bytes** of external EEPROM content

Example — fetch the 16 bytes read by the previous `cmdReadEXTEEPROM` :

```
Host → MCU:  AA 25 00 00 10
MCU  → Host:  25 10                                  (acknowledge)
MCU  → Host:  00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF   (16 bytes)
```

`0x26` — Program Sensor EEPROM (`cmdProgramEEPROM`)

Programs the sensor EEPROM with 18 × 16-bit words (36 bytes). Applies the required programming voltage sequence internally.

- **Send**: `[0xAA][0x26][SENSOR_SELECT][0x24][N_RESP]` followed by **36 payload bytes** = 18 words, each **MSB first**: `[W0_H][W0_L][W1_H][W1_L] ... [W17_H][W17_L]`
- **Reply**: Intermediate acknowledge `[0x26][N_RESP]` , then final acknowledge `[data[1]][N_RESP]`

> **WARNING — CRC is not recalculated.** This command programs the 36-byte payload into the sensor EEPROM **exactly as supplied**. The firmware performs **no** CRC calculation, patching, or validation. The host must recalculate the CRC over the complete modified 18-word image and place it in **word 2, bits 7:0** before sending. Programming an image with an incorrect CRC leaves the sensor in a permanent fault state (OCD outputs driven to GND) after the next start-up. See Important: EEPROM CRC Handling.

Example — program Sensor 1 with 18 words (word 0 = `0x1A2B` , word 1 = `0x0004` , …):

```
Host → MCU:  AA 26 00 24 00
MCU  → Host:  26 00                                  (intermediate ack)
Host → MCU:  1A 2B 00 04 ... (36 bytes total)        (18 words, MSB first)
MCU  → Host:  2B 00                                  (final ack, CMD_ECHO = data[1] = 0x2B)
```

`0x27` — Check External EEPROM Presence (`cmdCheckIfExtEEPROMIsPresent`)

Detects whether the external I²C EEPROM responds on the bus.

- **Send**: `[0xAA][0x27][SENSOR_SELECT][0x00][0x02]`
- **Reply**:
  1. Acknowledge frame `[0x27][0x02]`
  2. **2 data bytes**: `[0x01][0x01]` if present, `[0x00][0x00]` if absent

> If the EEPROM is not detected, the MCU performs a system reset after replying.

Example — check for the external EEPROM (present):

```
Host → MCU:  AA 27 00 00 02
MCU  → Host:  27 02                                  (acknowledge)
MCU  → Host:  01 01                                   (present; 00 00 would mean absent)
```

`0x28` — Read Temperature (`cmdReadTemperature`)

Reads the temperature register (register `24`) of all three sensors. Each sensor is briefly placed in test mode to perform the read.

- **Send**: `[0xAA][0x28][SENSOR_SELECT][0x00][0x06]`
- **Reply**:
  1. Acknowledge frame `[0x28][0x06]`
  2. **6 data bytes**, each sensor as a 16-bit value **MSB first**: `[T1_H][T1_L][T2_H][T2_L][T3_H][T3_L]`

Example — read all three temperatures (T1 = `0x0320`, T2 = `0x0318`, T3 = `0x0325`):

```
Host → MCU:  AA 28 00 00 06
MCU  → Host:  28 06                                  (acknowledge)
MCU  → Host:  03 20 03 18 03 25                       (T1, T2, T3 raw register values)
```

`0x29` — Enable VREF Multiplexer (`cmdEnableMux`)

Enables/disables the VREF reference multiplexer and selects the reference level.

- **Send**: `[0xAA][0x29][SENSOR_SELECT][0x02][N_RESP]` followed by 2 payload bytes:

Payload byte	Field	Description
`data[0]`	Enable	`1` = enable mux, `0` = disable mux (all lines low)
`data[1]`	Level select	When enabled: `1` selects the reference on all sensor VREF lines; `0` leaves lower reference disabled

- **Reply**: Intermediate acknowledge `[0x29][N_RESP]`, then final acknowledge `[data[1]][N_RESP]`

> **Note**: The command drives the shared VREF mux enable line and all three per-sensor VREF mux lines together. It ignores `SENSOR_SELECT`; the evaluation software always sends `0x00`.

> **Part of the readout start-up sequence.** This command must be issued after `cmdResetSensor` (`0x13`) and before `cmdStartReadout` (`0x14`). Use `01 01` when the sensor's EEPROM `OPMODE` is single-ended (`3`), and `00 00` for all differential modes (`0`, `1`, `2`). See Required Start-Up Sequence.

Example — enable the VREF mux and select the reference:

```
Host → MCU:  AA 29 00 02 00
MCU  → Host:  29 00                                  (intermediate ack)
Host → MCU:  01 01                                   (payload: enable, level select)
MCU  → Host:  01 00                                  (final ack, CMD_ECHO = data[1] = 0x01)
```

`0x30` — Write Register (`cmdSetRegister`)

Writes a 16-bit value to a sensor register over SICI.

- **Send**: `[0xAA][0x30][SENSOR_SELECT][0x03][N_RESP]` followed by 3 payload bytes:

Payload byte	Field	Description
`data[0]`	Register address	Address of the register to write
`data[1]`	Value MSB	High byte of the 16-bit value
`data[2]`	Value LSB	Low byte of the 16-bit value (`value = (data[1] << 8) + data[2]`)

- **Reply**: Intermediate acknowledge `[0x30][N_RESP]`, then final acknowledge `[data[1]][N_RESP]`

> **WARNING — CRC is not recalculated.** The written value is taken verbatim; no CRC is computed or checked. When the target address belongs to the EEPROM address space (`0x40` – `0x51`), writing a single word **invalidates the stored CRC**. The host must read the full image, apply all modifications, recalculate the CRC, and write the corrected value into word 2 bits 7:0 as part of the same programming sequence. See Important: EEPROM CRC Handling.

Example — write `0xABAC` to register `0x10` of Sensor 1:

```
Host → MCU:  AA 30 00 03 00
MCU  → Host:  30 00                                 (intermediate ack)
Host → MCU:  10 AB AC                               (payload: register, value MSB, value LSB)
MCU  → Host:  AB 00                                 (final ack, CMD_ECHO = data[1] = 0xAB)
```

`0x31` — Calibrate Programmer Board (`cmdCalibrateBoard`)

Reads the on-board VSENS calibration ADC channel, accumulating 4000 samples, and returns the 24-bit sum.

- **Send**: `[0xAA][0x31][SENSOR_SELECT][0x00][0x03]`
- **Reply**:
  1. Acknowledge frame `[0x31][0x03]`
  2. **3 data bytes** = 24-bit accumulated ADC sum, **MSB first**: `[SUM_23_16][SUM_15_8][SUM_7_0]`

Example — run board calibration (sum = `0x1E8480`):

```
Host → MCU:  AA 31 00 00 03
MCU  → Host:  31 03                                 (acknowledge)
MCU  → Host:  1E 84 80                              (24-bit ADC sum = 0x1E8480)
```

`0x32` — Write Flash Sector (`cmdWriteFlashSector`)

Writes a data block to the reserved application flash sector (virtual FAT block at address `0x0C1F0000`). Up to 14 payload bytes are stored.

- **Send**: `[0xAA][0x32][SENSOR_SELECT][N_DATA][N_RESP]` followed by up to 14 payload bytes (`data[0..13]`).
- **Reply**: Intermediate acknowledge `[0x32][N_RESP]` , then final acknowledge `[data[1]][N_RESP]`

> **Note — no checksum is generated.** The block is stored in MCU flash verbatim; the firmware neither appends nor verifies a checksum. This sector is unrelated to the sensor EEPROM CRC. If the host stores sensor configuration data here, it must maintain its own integrity check and, for any image later programmed into a sensor, a valid sensor CRC. See Important: EEPROM CRC Handling.

Example — write a 14-byte block to the reserved flash sector:

```
Host → MCU:  AA 32 00 0E 00
MCU  → Host:  32 00                                 (intermediate ack)
Host → MCU:  0F 19 23 2D 00 01 02 03 04 05 06 07 08 09   (14 payload bytes)
MCU  → Host:  19 00                                 (final ack, CMD_ECHO = data[1] = 0x19)
```

`0x33` — Read Flash Sector (`cmdReadFlashSector`)

Reads back the first 14 bytes of the reserved flash sector at address `0x0C1F0000` .

- **Send**: `[0xAA][0x33][SENSOR_SELECT][0x00][0x0E]`
- **Reply**:
    1. Acknowledge frame `[0x33][0x0E]`
    2. **14 data bytes** of flash sector content

Example — read back the reserved flash sector:

```
Host → MCU:  AA 33 00 00 0E
MCU  → Host:  33 0E                                  (acknowledge)
MCU  → Host:  0F 19 23 2D 00 01 02 03 04 05 06 07 08 09   (14 bytes)
```

ADC Readout Stream Format

After a `cmdStartReadout` (`0x14`), the MCU continuously streams fixed **12-byte frames** at the ADC sampling rate until `cmdStopReadout` (`0x15`) is issued. Each frame packs the AOUT and VREF ADC results (12-bit) of all three sensors plus the over-current detection (OCD) pin states.

> **Reminder**: `cmdResetSensor` (`0x13`) must be issued before `cmdStartReadout`. Without it, any sensor left in SICI test mode has its ISM powered down and its AOUT/VREF fields in the stream are meaningless.

Frame Layout (12 bytes)

Offset	Field	Description
0	`0xAA`	Frame start marker
1	`S1_AOUT_L`	Sensor 1 AOUT, bits 7:0
2	`S1_VREF_L`	Sensor 1 VREF, bits 7:0
3	`S1_HI`	High nibbles: `((AOUT & 0xF00) >> 4) | ((VREF & 0xF00) >> 8)` — AOUT bits 11:8 in the upper nibble, VREF bits 11:8 in the lower nibble
4	`S2_AOUT_L`	Sensor 2 AOUT, bits 7:0
5	`S2_VREF_L`	Sensor 2 VREF, bits 7:0
6	`S2_HI`	Sensor 2 high nibbles (same packing as offset 3)
7	`S3_AOUT_L`	Sensor 3 AOUT, bits 7:0
8	`S3_VREF_L`	Sensor 3 VREF, bits 7:0
9	`S3_HI`	Sensor 3 high nibbles (same packing as offset 3)
10	`OCD`	Over-current detection pin states (see bit map below)
11	`CHECKSUM`	8-bit sum of bytes 1–10 (modulo 256)

Reconstructing 12-bit ADC Values

```
S1_AOUT = ((S1_HI & 0xF0) << 4) | S1_AOUT_L
S1_VREF = ((S1_HI & 0x0F) << 8) | S1_VREF_L
```

(and analogously for Sensor 2 and Sensor 3).

Worked example — decoding one 12-byte frame:

```
Frame:  AA  34 12 71  56 22 41  8A 05 30  C0  <CS>
        |   |  |  |   |  |  |   |  |  |   |
        |   S1 fields  S2 fields  S3 fields  OCD
        start
```

For Sensor 1: `S1_AOUT_L = 0x34`, `S1_VREF_L = 0x12`, `S1_HI = 0x71`:

```
S1_AOUT = ((0x71 & 0xF0) << 4) | 0x34 = (0x70 << 4) | 0x34 = 0x734  (1844)
S1_VREF = ((0x71 & 0x0F) << 8) | 0x12 = (0x01 << 8) | 0x12 = 0x112  (274)
```

The OCD byte `0xC0` = `1100 0000b` means Sensor 1 OCD_A = 1 and OCD_B = 1, all other OCD pins low.

OCD Byte (offset 10)

Bit	Signal
7	Sensor 1 OCD_A
6	Sensor 1 OCD_B
5	Sensor 2 OCD_A
4	Sensor 2 OCD_B
3	Sensor 3 OCD_A
2	Sensor 3 OCD_B
1–0	Reserved (0)

Checksum Validation

The receiver should validate each frame by summing bytes 1 through 10 (8-bit, wrapping) and comparing against byte 11. Frames failing the check, or lacking the `0xAA` start marker at offset 0, should be discarded and the stream resynchronized on the next `0xAA`.

Quick Reference: Command Codes

Hex	Dec	Command
`0x10`	16	Read Sensor EEPROM
`0x11`	17	Enter Test Mode
`0x13`	19	Reset Sensor
`0x14`	20	Start Readout
`0x15`	21	Stop Readout
`0x16`	22	Oscilloscope / Trigger Config
`0x17`	23	Read Register
`0x18`	24	Get Register Value
`0x19`	25	Reset to Bootloader
`0x21`	33	Firmware Version
`0x22`	34	Initialize MCU
`0x23`	35	Program External EEPROM
`0x24`	36	Read External EEPROM
`0x25`	37	Get External EEPROM Value
`0x26`	38	Program Sensor EEPROM
`0x27`	39	Check External EEPROM Presence
`0x28`	40	Read Temperature
`0x29`	41	Enable VREF Multiplexer
`0x30`	48	Write Register
`0x31`	49	Calibrate Board
`0x32`	50	Write Flash Sector
`0x33`	51	Read Flash Sector

Complete Programming Example

This section walks through a complete, realistic session: connecting to the board, entering firmware mode, reading the EEPROM of Sensor 1, changing a single configuration field, recalculating the CRC, programming, and verifying the result.

The example data corresponds to a TLE4972 configured for **fully-differential output, measurement range S5**, and changes only the **OCD2 threshold from 21 to 23**. Everything else in the image is left untouched.

> **Scope**: All sensor operations below target **Sensor 1** (`SENSOR_SELECT = 0x00`). The same sequence applies to Sensor 2 (`0x01`) and Sensor 3 (`0x02`).

> **WARNING**: Step 6 is mandatory. Skipping the CRC recalculation programs an image that the sensor rejects at its next start-up, driving the OCD outputs to GND.

Notation

All console transcripts below use the following convention:

```
TX  <bytes>     host  -> MCU
RX  <bytes>     MCU   -> host
```

Long payloads are printed **8 bytes per line**, which for the 36-byte EEPROM stream maps to exactly 4 EEPROM words per line.

Step 1 — Connect the Board and Configure the Serial Port

Connect the programmer to the PC via USB, verify board power, then open the enumerated virtual COM port with the settings below (identical for bootloader and firmware mode).

Parameter	Value
Baud rate	1,250,000
Data bits	8
Stop bits	1
Parity	None
Flow control	None

```
[host] scanning ports...
[host] found FTDI device on COM7
[host] opening COM7 @ 1250000 8N1, no flow control
[host] port open
```

Step 2 — Bootloader Handshake (Echo)

After power-on the MCU runs the bootloader. Validate the link with the echo command before doing anything else.

```
TX  09                              echo command
RX  20                              bootloader acknowledgment

[host] bootloader responding -> link OK
```

If no `0x20` is received, the link or the power supply is faulty; do not continue.

Step 3 — Switch to Firmware Mode

Command `0x12` finalizes the bootloader and restarts the MCU into the application firmware (ABM0, flash `0x08010000`).

```
TX  12                              exit bootloader / start firmware
RX  20                              acknowledgment (restart follows)

[host] waiting for firmware restart... 500 ms
[host] firmware mode active
```

Confirm the firmware is alive by reading its version:

```
TX  AA 21 00 00 02                  cmdFwVersion
RX  21 02                           acknowledge
RX  00 32                           version = 0x32 = 50

[host] firmware version 50
```

> **Note**: The firmware image itself must be updated with the **Evaluation Kit Software**. This document does not cover the page-programming sequence beyond the command list in Bootloader Commands.

Step 4 — Initialize the MCU (Default Zero Mapping)

`cmdInitMCU` with array config `0x00` selects the default 1:1 slot mapping — logical Sensor 1 = Slot A, Sensor 2 = Slot B, Sensor 3 = Slot C.

```
TX  AA 22 00 02 00                  cmdInitMCU, 2 payload bytes
RX  22 00                           intermediate ack
TX  00 00                           payload: array config = 0x00, echo byte = 0x00
RX  00 00                           final ack (CMD_ECHO = data[1] = 0x00)

[host] MCU initialized, array config 0 (default mapping)
```

Step 5 — Put Sensor 1 into Test Mode

SICI access requires the sensor to be in test mode. This command sends the interface password and powers down the ISM.

```
TX  AA 11 00 00 00                    cmdEnterTestMode, sensor 1
RX  11 00                             acknowledge

[host] sensor 1 in SICI test mode, ISM powered down
```

Step 6 — Read the Full EEPROM of Sensor 1

The complete 18-word image must be read before any modification, because the CRC covers all words.

```
TX  AA 10 00 00 24                    cmdReadEEPROM, expect 36 bytes
RX  10 24                             acknowledge
RX  C0 30 57 47 00 FE F5 E6           words 0..3
RX  F3 1D 0B E1 49 9D 45 85           words 4..7
RX  00 07 00 F7 00 23 FE F2           words 8..11
RX  BC 30 F7 C2 44 CF 1C 70           words 12..15
RX  FE 65 DB 40                       words 16..17

[host] EEPROM read OK (36 bytes)
[host] image: C030 5747 00FE F5E6 F31D 0BE1 499D 4585
[host]        0007 00F7 0023 FEF2 BC30 F7C2 44CF 1C70
[host]        FE65 DB40
[host] CRC stored = 0xFE, CRC calculated = 0xFE -> VALID
```

Decoded configuration of the user-accessible words:

Word	Address	Value	Decoded
0	`0x40`	`0xC030`	`OCD2EN=1`, `OCD1EN=1`, `OCD2DEGLITCH=0`, `OCD1DEGLITCH=0`, `OPMODE=1` (fully-differential), `MEASRNG=0x10` (S5)
1	`0x41`	`0x5747`	`OCD2THRSH=21`, `OCD2HYST=3`, `OCD1THRSH=17`, `OCD1HYST=3`
2	`0x42`	`0x00FE`	`CRC=0xFE`, `VREFEXT=0` (1.65 V), all option bits 0
3–17	`0x43` – `0x51`	—	Factory calibration coefficients — **do not modify**

Step 7 — Modify the OCD2 Threshold (21 → 23)

`OCD2THRSH` occupies **bits 15:10 of word 1**. Only those six bits change; the remaining bits of word 1 and every other word are preserved.

```
word[1] = (word[1] & 0x03FF) | (23u << 10);
```

```
  old word 1 : 0x5747  =  010101 11 010001 11
                          ^^^^^^ OCD2THRSH = 21
  new word 1 : 0x5F47  =  010111 11 010001 11
                          ^^^^^^ OCD2THRSH = 23
```

```
[host] modifying word 1: 0x5747 -> 0x5F47 (OCD2THRSH 21 -> 23)
[host] all other words unchanged
```

At this point the image in RAM is **inconsistent** — the stored CRC `0xFE` no longer matches the payload.

Step 8 — Recalculate the CRC and Patch the Stream

Run `eepromCrcCalc()` from Reference Implementation © over the **modified** image, then write the result into word 2, bits 7:0.

```
uint16_t eeprom[18];                /* image read in step 6, word 1 already modified */

uint8_t crc = eepromCrcCalc(eeprom); /* -> 0xF2 */
eeprom[2] = (uint16_t)((eeprom[2] & 0xFF00u) | crc);

if (!eepromCrcCheck(eeprom))        /* sanity check before programming */
{
    /* abort - do not program */
}
```

```
[host] recalculating CRC over modified image...
[host] CRC old = 0xFE, CRC new = 0xF2
[host] patching word 2: 0x00FE -> 0x00F2
[host] self-check: eepromCrcCheck() = 1 -> image consistent
```

Resulting 36-byte stream to program (changed bytes marked):

```
C0 30 5F 47 00 F2 F5 E6               words 0..3   (^^ word1, ^^ word2 CRC)
F3 1D 0B E1 49 9D 45 85               words 4..7
00 07 00 F7 00 23 FE F2               words 8..11
BC 30 F7 C2 44 CF 1C 70               words 12..15
FE 65 DB 40                           words 16..17
```

Word	Before	After
Word 1 (`0x41`)	`0x5747`	`0x5F47`
Word 2 (`0x42`)	`0x00FE`	`0x00F2`

Step 9 — Program the New EEPROM Image

`cmdProgramEEPROM` takes the 36-byte stream verbatim and runs the full programming voltage sequence internally.

```
TX  AA 26 00 24 00                    cmdProgramEEPROM, 36 payload bytes
RX  26 00                             intermediate ack
TX  C0 30 5F 47 00 F2 F5 E6           words 0..3
TX  F3 1D 0B E1 49 9D 45 85           words 4..7
TX  00 07 00 F7 00 23 FE F2           words 8..11
TX  BC 30 F7 C2 44 CF 1C 70           words 12..15
TX  FE 65 DB 40                       words 16..17
RX  30 00                             final ack (CMD_ECHO = data[1] = 0x30)

[host] EEPROM programming sequence complete
```

> The final acknowledge echoes payload byte `data[1]` (`0x30`, the low byte of word 0) instead of the command code — this is the documented `CMD_ECHO` behavior for payload-carrying commands.

Step 10 — Power-Cycle the Sensor

The new content only becomes effective after a sensor restart, and the internal CRC check runs at start-up.

```
TX  AA 13 00 00 00                    cmdResetSensor, sensor 1
RX  13 00                             acknowledge

[host] sensor 1 VDD off 100 ms, on
[host] sensor restarted
```

Step 11 — Read Back and Verify

Re-enter test mode and read the full image again.

```
TX  AA 11 00 00 00                    cmdEnterTestMode, sensor 1
RX  11 00                             acknowledge

TX  AA 10 00 00 24                    cmdReadEEPROM, expect 36 bytes
RX  10 24                             acknowledge
RX  C0 30 5F 47 00 F2 F5 E6           words 0..3
RX  F3 1D 0B E1 49 9D 45 85           words 4..7
RX  00 07 00 F7 00 23 FE F2           words 8..11
RX  BC 30 F7 C2 44 CF 1C 70           words 12..15
RX  FE 65 DB 40                       words 16..17
```

Host-side verification:

```
uint16_t readback[18];              /* image from the read-back above */

bool match = (memcmp(readback, eeprom, sizeof(eeprom)) == 0);
bool crcOk = eepromCrcCheck(readback);
```

```
[host] read-back compare : MATCH (36/36 bytes)
[host] CRC stored = 0xF2, CRC calculated = 0xF2 -> VALID
[host] word 1 = 0x5F47 -> OCD2THRSH = 23 (was 21)
[host] PROGRAMMING SUCCESSFUL
```

A useful final sanity check is to confirm the sensor is not in fault state — with a valid CRC the OCD outputs are released, which is visible in the OCD byte of the readout stream.

The sensor is still in SICI test mode after the read-back, so the full Required Start-Up Sequence must be executed first. This example device runs in **fully-differential** mode (`OPMODE = 1`), so the VREF mux is **disabled**.

```
TX  AA 13 00 00 00                    1. cmdResetSensor - mandatory before readout
RX  13 00                             acknowledge

[host] sensor supply cycled, waiting 100 ms
[host] sensors in normal operating mode

TX  AA 29 00 02 00                    3. cmdEnableMux
RX  29 00                             intermediate ack
TX  00 00                             payload: disable, level 0 (OPMODE = 1, not single-ended)
RX  00 00                             final ack (CMD_ECHO = data[1] = 0x00)

[host] VREF mux disabled (differential mode)

TX  AA 14 00 00 00                    4. cmdStartReadout
RX  14 00                             acknowledge
RX  AA 34 12 71 56 22 41 8A           12-byte frame (bytes 0..7)
RX  05 30 00 2F                       12-byte frame (bytes 8..11), OCD = 0x00
               ^^ OCD byte = 0x00 -> no OCD asserted, no CRC fault
                  ^^ checksum = sum(bytes 1..10) & 0xFF = 0x2F

TX  AA 15 00 00 00                    cmdStopReadout
RX  15 00                             acknowledge
```

> If the programmed image had used single-ended mode (`OPMODE = 3`), the mux payload would be `01 01` instead of `00 00`.

Sequence Summary

#	Action	Frame
1	Open COM port @ 1,250,000 8N1	—
2	Bootloader echo	`09` → `20`
3	Switch to firmware	`12` → `20`
3b	Firmware version	`AA 21 00 00 02`
4	Init MCU, default mapping	`AA 22 00 02 00` + `00 00`
5	Sensor 1 test mode	`AA 11 00 00 00`
6	Read EEPROM	`AA 10 00 00 24`
7	Modify word 1 (host side)	—
8	**Recalculate CRC, patch word 2 (host side)**	—
9	Program EEPROM	`AA 26 00 24 00` + 36 bytes
10	Power-cycle sensor	`AA 13 00 00 00`
11	Test mode + read back + verify	`AA 11 00 00 00`, `AA 10 00 00 24`
12	**Reset sensor (mandatory before readout)**	`AA 13 00 00 00`
12b	Wait 100 ms	—
12c	**Configure VREF mux for the output mode**	`AA 29 00 02 00` + `00 00` (or `01 01` if single-ended)
13	Start readout, check OCD, stop	`AA 14 00 00 00`, `AA 15 00 00 00`

Double Code Word Calibration Flow

This chapter maps the **Double Code Word (DCW)** single-point calibration method of the TLE4972 user manual (Rev. 1.0.1, chapters 6.4, 6.6.2, 6.6.2.1 and 6.6.2.2) onto the serial commands of the programmer firmware. It describes *how to drive the procedure over the serial link*; the underlying theory, accuracy considerations and the applicability of the method remain the subject of the sensor user manual.

> **WARNING — this procedure rewrites factory calibration data.** The coefficients `o_base`, `g_base`, `g_tc_tl`, `g_tc_tq` and `g_tc_tt` are determined per device on the production line over the full temperature range. A DCW calibration replaces them with values derived from a **single room-temperature measurement**. Always store the complete original 18-word EEPROM image *before* the first write, and keep it until the calibration has been verified. Every EEPROM write additionally requires a recalculated CRC — see Important: EEPROM CRC Handling.

Method Summary

The method exploits two linear relationships:

Relationship	Swept register	Sweep values	Measured quantity
`Offset_CW` ↔ output offset	Offset Code Word	`Offset_CW1 = -40`, `Offset_CW2 = +40`	`Offset1`, `Offset2` at zero primary current
`Gain_CW` ↔ **inverse** sensitivity	Gain Code Word	`Gain_CW1 = 210`, `Gain_CW2 = 570`	`Sensitivity1`, `Sensitivity2` with test current `ITEST`

Four measurements yield the two target code words by linear interpolation:

$$\text{Sensitivity}{*RATIO}=*frac{*frac{1}{S*}{\text{target}}-\frac{1}{S_1}}{\frac{1}{S_2}-\frac{1}{S_1}}\qquad Gain_CW_{target}=Gain_CW_1+\text{Sensitivity}_{RATIO}\times\left(Gain_CW_2-Gain_CW_1\right)$$

$$Offset_{AVG}=\frac{O_1+O_2}{2}\qquad Offset_{RATIO}=\frac{O_2-O_1}{Offset_CW_2-Offset_CW_1}\qquad Offset_CW_{target}=-\frac{Offset_{AVG}}{Offset_{RATIO}}$$

The two target code words are then converted into new EEPROM calibration coefficients (see Phase 4) and programmed.

Registers and EEPROM Fields Involved

Internal sensor registers (accessed with `0x17` / `0x18` and `0x30`)

Address	Name	Implemented bits	Contents	Access note
`0x18`	Internal temperature	15:0	`TINT`, unsigned ADC code	ISM must be powered down
`0x20`	Gain Code Word	10:0	`Gain_CW`, unsigned, 0–2047	ISM must be powered down
`0x21`	Offset Code Word	7:0	`Offset_CW`, **sign-magnitude**: bit 7 = sign, bits 6:0 = magnitude, −127…+127	ISM must be powered down
`0x25`	SICI bypass	15, 12	bit 15 `test_pd_ism`, bit 12 `SICI_mode_dis`	See Measurement Mode
`0x16`	Diagnosis mode amplitude	15	`Diag_mode_amp`, 0 = reduced amplitude	Not used by this procedure

Each address line is 16 bits wide; bits outside the ranges above are not implemented, so **mask a read-back to the field width before comparing it with what was written**. The widths line up with the EEPROM fields the registers trim: `g_base` is an 11-bit value and `o_base` is 8-bit signed.

Temperature conversion (sensitivity 16 LSB/°C, 1408 ≙ 25 °C):

$$T[^\circ C]=\frac{TINT-2048}{16}+65 \qquad\Longleftrightarrow\qquad T_{code}=\left(T[^\circ C]-65\right)\times 16+2048$$

The temperature of all three sensors can also be fetched in one transaction with `cmdReadTemperature` (`0x28`), which returns register `0x18` of Sensor 1, 2 and 3.

EEPROM words touched by the calibration

The 36-byte payload of `cmdReadEEPROM` / `cmdProgramEEPROM` maps word index *n* to sensor EEPROM address `0x40 + n` .

Word	Address	Bit field	Width / type	Coefficient
2	`0x42`	bits 7:0	unsigned	**CRC** — must be recalculated
3	`0x43`	bits 3:0	—	`g_base` bits 3:0
3	`0x43`	bits 15:4	12-bit signed	`g_tc_tl`
4	`0x44`	bits 5:0	—	`g_base` bits 9:4
4	`0x44`	bits 15:6	10-bit signed	`g_tc_tq`
5	`0x45`	bit 4	—	`g_base` bit 10
5	`0x45`	bits 15:5	11-bit signed	`g_tc_tt`
8	`0x48`	bits 7:0	8-bit signed	`o_base`

`g_base` is an 11-bit value scattered over three words:

```
g_base = (uint16_t)(( eeprom[3]        & 0x000Fu)        /* bits  3:0  */
                  | ((eeprom[4] & 0x003Fu) << 4)         /* bits  9:4  */
                  | ((eeprom[5] & 0x0010u) << 6));       /* bit  10    */
```

> Words 3, 4, 5 and 8 also carry fields that are **not** part of this procedure (`s_base` in word 5, `epk_base` in word 8). Always read-modify-write these words on top of the image read in Phase 1; never assemble a full word from the calibration coefficients alone.

Measurement Mode: SICI Off, ISM Still Down

The DCW procedure needs a state that the normal readout start-up sequence does not produce: the **ISM stays powered down** (so the code word written over SICI is not overwritten by the firmware of the sensor) while the **AOUT buffer runs in normal operating mode** (so AOUT and VREF can be measured).

Command	Effect on register `0x25`	Resulting state
`cmdEnterTestMode` (`0x11`)	bit 15 `test_pd_ism` set	SICI active, ISM down. AOUT carries the SICI signalling — **not measurable**
`cmdSetRegister` (`0x30`) with `25 90 00`	bit 15 kept, bit 12 `SICI_mode_dis` set	SICI disabled, ISM still down, AOUT buffer in normal operating mode — **measurable**

> **Exception to the Required Start-Up Sequence.** During the DCW sweeps you must **not** issue `cmdResetSensor` (`0x13`) before `cmdStartReadout` (`0x14`): the reset would power-cycle the part and discard the code word just written. Configure the VREF mux with `cmdEnableMux` (`0x29`) according to `OPMODE` as usual, then start the readout directly.

> **One-way door.** Setting `SICI_mode_dis` disables the SICI interface until the next chip reset. Each of the four measurement points therefore needs its own power cycle → test mode → register write → SICI disable → measure cycle. This is why `cmdResetSensor` appears at the top of every loop iteration in the flowchart of the user manual.

> **`cmdEnterTestMode` already power-cycles the part.** The firmware implementation drives the OCD and SICI lines low, removes VDD for 100 ms, restores it and only then performs the timed SICI entry with the password. A separate `cmdResetSensor` before it is redundant, and it power-cycles with the pins floating and AOUT actively driven, which can keep the die parasitically powered so that `SICI_mode_dis` never clears. Go straight to `cmdEnterTestMode`.

> **Verify that SICI actually came up.** After `cmdEnterTestMode`, probe the link by reading the internal temperature register `0x18` and decoding it with `T[°C] = (TINT − 2048)/16 + 65`. A value inside the operating range of the device proves that SICI answers; anything outside it means the programmer is bit-banging into a device that is not listening, so every subsequent read returns noise and every write is silently lost. A code word read-back such as *wrote* `0x023A`, *read* `0x22F8` is this failure, not a field-width problem. Retry the entry before trusting any register access.
>
> Do **not** probe with register `0x25`. It is a write-only test register: after `cmdEnterTestMode` it reads back something like `0x0078`, not the `0x8000` that was written into it. The user manual only ever describes *setting* its bits, never reading them. The registers documented as readable are `0x18` (`TINT`), `0x20` (`Gain_CW`) and `0x21` (`Offset_CW`) — chapter 6.6.2.2 explicitly requires reading the latter two back as `Gain_CW_old` and `Offset_CW_old`.

> **Expect the SICI link to be occasionally flaky.** SICI entry depends on a hard-coded timing window in the programmer firmware (`WaitUs(1350) + WaitUs(1500)` after VDD returns), and single transfers do garble. Do not abort the calibration on the first bad read-back: re-issue the `cmdSetRegister` a couple of times, and if the code word still will not confirm, power-cycle with `cmdEnterTestMode` and start the measurement point over. Only give up after a few full retries — a persistent failure points at the AOUT/SICI wiring, the ground return or the supply decoupling.

> **Drain the readout stream before the next command.** `cmdStopReadout` does not stop the device instantly: its 2-byte acknowledge is interleaved with stream frames, and further frames keep arriving for a short while. A single fixed-delay flush is not enough — anything that lands after it stays in the receive buffer, and the next command's acknowledge read consumes those stale bytes instead. From then on the whole byte stream is shifted and every register read returns nonsense, with a *different* wrong value each time. A read-back containing `0xAA` (for example `0xFCAA`) is the give-away: that is a stream frame start byte sitting in a data position. Read and discard until the line has stayed quiet for a while, and re-check before every command frame.

> **Re-read, then re-program, before declaring a programming failure.** `cmdReadEEPROM` returns all 36 bytes in one burst, and `burnEEPROM()` pushes the 18 words to the sensor as 18 unverified SICI transfers. Neither side checks itself, so a single garbled word is expected occasionally — and a read-back where 17 of 18 words match is exactly that. It is *not* a byte-shifted stream (that would corrupt every word from the shift onwards) and *not* a failed erase (that would leave a whole region wrong). Read the image again a couple of times to rule out a read glitch, and if the same word still differs, the write itself was corrupted: burn the image again. Keep the retry count small — the EEPROM is rated for 100 programming cycles.

> **Settling.** Wait at least `TISM_SETTLING` (max. 100 µs) after the state change before sampling. In practice the sampling latency of the readout stream already exceeds this.

Measuring Offset and Sensitivity

Both quantities are derived from the differential output `VO = V(AOUT) - V(VREF)`, reconstructed from the 12-byte readout frames (see ADC Readout Stream Format):

```
VO_code = S1_AOUT - S1_VREF        (per frame, 12-bit codes)
```

Quantity	Primary current	Formula	Manual reference
Offset	0 A	`Offset = VO(I = 0)`	ch. 6.4, offset bullets
Sensitivity	0 A **and** `ITEST`	`Sensitivity = (VO(ITEST) - VO(I = 0)) / ITEST`	ch. 6.4, eq. (8)

> Sensitivity is a **slope**, so every gain sweep point needs its own zero-current reference: one measurement at `ITEST` cannot separate the sensitivity from the offset. The zero is not reusable between `Gain_CW1` and `Gain_CW2` because the gain stage also scales the input-referred offset. Measure the 0 A point first, before the conductor heats up.

- Average **at least 100 frames** per measurement point (user manual ch. 6.4). The calibration is a DC measurement; all AC components must be filtered out.
- `ITEST` must be at least 10 % of the target full-scale current `IFS`, and low enough to avoid self-heating during the calibration.
- `ITEST` itself has to be measured with a calibrated source, shunt or current probe — the programmer cannot supply this value.
- For a ratiometric part, measure `VDD` as well and apply the ratiometric correction of user manual eq. (9), `Sensitivity x 3.3 V / VDD`. The averaged on-board VSENS channel returned by `cmdCalibrateBoard` (`0x31`) can serve as this supply reference.

Command Building Blocks

Flowchart action	Serial command	Frame
Power cycle the device	`cmdResetSensor` (`0x13`)	`AA 13 00 00 00`
Enter SICI and disable the ISM	`cmdEnterTestMode` (`0x11`)	`AA 11 00 00 00`
Store the original EEPROM content	`cmdReadEEPROM` (`0x10`)	`AA 10 00 00 24`
Read `Gain_CW` / `Offset_CW` / `TINT`	`cmdReadRegister` (`0x17`) + `cmdGetRegisterValue` (`0x18`)	`AA 17 00 02 00` + `00 <addr>`, then `AA 18 00 00 02`
Read the internal temperature of all sensors	`cmdReadTemperature` (`0x28`)	`AA 28 00 00 06`
Set `Gain_CW` / `Offset_CW`	`cmdSetRegister` (`0x30`)	`AA 30 00 03 00` + `<addr> <MSB> <LSB>`
Disable SICI, keep the ISM down	`cmdSetRegister` (`0x30`)	`AA 30 00 03 00` + `25 90 00`
Configure the VREF reference	`cmdEnableMux` (`0x29`)	`AA 29 00 02 00` + `00 00` or `01 01`
Measure AOUT / VREF	`cmdStartReadout` (`0x14`) … `cmdStopReadout` (`0x15`)	`AA 14 00 00 00` … `AA 15 00 00 00`
Program the new coefficients	`cmdProgramEEPROM` (`0x26`)	`AA 26 00 24 00` + 36 bytes

Worked Example

All frames below use Sensor 1 (`SENSOR_SELECT = 0x00`) and the notation of the Complete Programming Example. The numeric values are illustrative; the real ones come from the device under calibration and from the measurement setup.

Assumptions for the example: target sensitivity `S_target = 5.000 mV/A`, part in fully-differential output mode (`OPMODE = 1` , mux payload `00 00`).

Phase 1 — Back Up the Device State

```
TX  AA 13 00 00 00                    cmdResetSensor (defined starting point)
RX  13 00

TX  AA 11 00 00 00                    cmdEnterTestMode (SICI on, ISM down)
RX  11 00

TX  AA 10 00 00 24                    cmdReadEEPROM
RX  10 24                             acknowledge
RX  C0 30 57 47 00 FE F5 E6           words 0..3
RX  F3 1D 0B E1 49 9D 45 85           words 4..7
RX  00 07 00 F7 00 23 FE F2           words 8..11
RX  BC 30 F7 C2 44 CF 1C 70           words 12..15
RX  FE 65 DB 40                       words 16..17

[host] ORIGINAL IMAGE SAVED TO FILE - required for rollback
[host] o_base_old  = 0x07 =   7          (word 8, bits 7:0, signed)
[host] g_base_old  =       470          (word 3 bits 3:0 | word 4 bits 5:0 | word 5 bit 4)
[host] g_tc_tl_old = 0xF5E = -162       (word 3, bits 15:4, 12-bit signed)
[host] g_tc_tq_old = 0x3CC =  -52       (word 4, bits 15:6, 10-bit signed)
[host] g_tc_tt_old = 0x05F =   95       (word 5, bits 15:5, 11-bit signed)
[host] s_base_3_0  = 0x1                (word 5, bits 3:0 - NOT part of this procedure)

TX  AA 17 00 02 00                    cmdReadRegister -> Gain_CW
RX  17 00                             intermediate ack
TX  00 20                             payload: reserved, address 0x20
RX  20 00                             final ack (CMD_ECHO = 0x20)

TX  AA 18 00 00 02                    cmdGetRegisterValue
RX  18 02                             acknowledge
RX  01 56                             Gain_CW_old = 0x0156 = 342

TX  AA 17 00 02 00                    cmdReadRegister -> Offset_CW
RX  17 00
TX  00 21                             payload: reserved, address 0x21
RX  21 00

TX  AA 18 00 00 02                    cmdGetRegisterValue
RX  18 02
RX  00 03                             Offset_CW_old = +3 (sign-magnitude)

TX  AA 17 00 02 00                    cmdReadRegister -> internal temperature
RX  17 00
TX  00 18                             payload: reserved, address 0x18
RX  18 00

TX  AA 18 00 00 02                    cmdGetRegisterValue
RX  18 02
RX  05 80                             TINT = 1408 -> TCAL = 25.0 C
```

> `Offset_CW` is an 8-bit **sign-magnitude** field: bit 7 is the sign, bits 6:0 the magnitude, so −40 is written as `0xA8` and +40 as `0x28`. Mask the read-back to 8 bits and confirm it before relying on a measurement.

Phase 2 — Gain Sweep (`Gain_CW1 = 210` , `Gain_CW2 = 570`)

Repeat the block below for `i = 1, 2` with `Gain_CW1 = 210 = 0x00D2` and `Gain_CW2 = 570 = 0x023A` .

```
TX  AA 13 00 00 00                    1. cmdResetSensor (power cycle)
RX  13 00
                                      ... wait 100 ms ...

TX  AA 11 00 00 00                    2. cmdEnterTestMode (SICI on, ISM down)
RX  11 00

TX  AA 30 00 03 00                    3. cmdSetRegister -> Gain_CW
RX  30 00                             intermediate ack
TX  20 00 D2                          payload: address 0x20, value 0x00D2 (210)
RX  00 00                             final ack (CMD_ECHO = data[1] = 0x00)

TX  AA 30 00 03 00                    4. cmdSetRegister -> SICI bypass
RX  30 00
TX  25 90 00                          payload: address 0x25, value 0x9000
RX  90 00                             final ack (CMD_ECHO = data[1] = 0x90)

[host] SICI disabled, ISM still down, AOUT buffer in normal operating mode
[host] wait >= TISM_SETTLING (100 us)

TX  AA 29 00 02 00                    5. cmdEnableMux (per OPMODE, NO reset here)
RX  29 00
TX  00 00                             payload: disable, level 0 (differential mode)
RX  00 00

TX  AA 14 00 00 00                    6. cmdStartReadout
RX  14 00
RX  AA 34 12 71 56 22 41 8A           ... collect >= 100 frames at I = 0 ...
RX  05 30 00 2F
       ... apply ITEST, collect >= 100 more frames ...
TX  AA 15 00 00 00                    7. cmdStopReadout
RX  15 00

[host] i = 1: Gain_CW = 210 -> Sensitivity1 = 4.820 mV/A
[host] i = 2: Gain_CW = 570 -> Sensitivity2 = 5.310 mV/A
```

Interpolation on the **inverse** sensitivity:

```
1/S1 = 0.2074689   1/S2 = 0.1883239   1/S_target = 0.2000000
Sensitivity_RATIO = (0.2000000 - 0.2074689) / (0.1883239 - 0.2074689) = 0.390122
Gain_CW_target   = 210 + 0.390122 * (570 - 210) = 350.44 -> 350 (0x015E)
```

Phase 3 — Offset Sweep (`Offset_CW1 = -40` , `Offset_CW2 = +40`)

Identical to Phase 2, except that **both** code words are written before disabling SICI: `Offset_CW` gets the sweep value and `Gain_CW` gets `Gain_CW_target` from Phase 2. Measurements are taken at **zero** primary current.

```
TX  AA 13 00 00 00                    1. cmdResetSensor
RX  13 00
                                      ... wait 100 ms ...
TX  AA 11 00 00 00                    2. cmdEnterTestMode
RX  11 00

TX  AA 30 00 03 00                    3. cmdSetRegister -> Gain_CW = Gain_CW_target
RX  30 00
TX  20 01 5E                          payload: address 0x20, value 0x015E (350)
RX  01 00                             final ack (CMD_ECHO = data[1] = 0x01)

TX  AA 30 00 03 00                    4. cmdSetRegister -> Offset_CW = -40
RX  30 00
TX  21 80 28                          payload: address 0x21, sign bit + magnitude 0x28
RX  80 00                             final ack (CMD_ECHO = data[1] = 0x80)

TX  AA 30 00 03 00                    5. cmdSetRegister -> SICI bypass
RX  30 00
TX  25 90 00                          payload: address 0x25, value 0x9000
RX  90 00

TX  AA 29 00 02 00                    6. cmdEnableMux (per OPMODE)
RX  29 00
TX  00 00
RX  00 00

TX  AA 14 00 00 00                    7. cmdStartReadout, average >= 100 frames at I = 0
RX  14 00
      ...
TX  AA 15 00 00 00                    8. cmdStopReadout
RX  15 00

[host] j = 1: Offset_CW = -40 -> Offset1 = -5.400 mV
[host] j = 2: Offset_CW = +40 -> Offset2 = +3.800 mV
```

Interpolation to null offset:

```
Offset_AVG       = (-5.400 + 3.800) / 2                = -0.800 mV
Offset_RATIO     = (3.800 - (-5.400)) / (40 - (-40))   =  0.115 mV/LSB
Offset_CW_target = -(-0.800) / 0.115 = 6.96 -> 7
```

Phase 4 — Recalculate the EEPROM Coefficients

All temperatures are expressed as the internal temperature code $T=\left(T[^\circ C]-65\right)\times 16+2048$.

Step 1 — new base offset (user manual eq. 12):

$$o_base_{new}=o_base_{old}+Offset_CW_{target}-Offset_CW_{old}=7+7-3=11=\texttt{0x0B}$$

> The whole of Phase 4 is implemented in `compute_new_coefficients()` of TLE4972_DCW_CALIBRATION_EXAMPLE.py; the numbers in this chapter are that function's output for the example image.

Step 2 — internal gain and gain correction factor (eq. 14–16):

$$Gain(T)=\frac{15\times 2^{13}}{300\times\left(Gain_CW(T)+512\right)}\qquad\Longleftrightarrow\qquad Gain_CW(T)=\frac{15\times 2^{13}}{300\times Gain(T)}-512$$

$$Gain_{old}(T_{CAL})=\frac{122880}{300\times(342+512)}=0.479625\qquad Gain_{new}(T_{CAL})=\frac{122880}{300\times(350+512)}=0.475174$$

$$Gain_CF=\frac{Gain_{new}(T_{CAL})}{Gain_{old}(T_{CAL})}=0.990719$$

Step 3 — old temperature coefficients (eq. 17–19):

$$BASE_{old}=g_base_{old}\quad TL_{old}=\frac{g_tc_tl_{old}}{2^{11}}\quad TQ_{old}=\frac{g_tc_tq_{old}}{2^{22}}\quad TT_{old}=\frac{g_tc_tt_{old}}{2^{35}}$$

$$Gain_CW_{old}(T)=\mathrm{ROUND}\left(BASE_{old}+TL_{old}T+TQ_{old}T^{2}+TT_{old}T^{3}\right)$$

Step 4 — new gain over temperature (eq. 20–21): apply `Gain_CF` to `Gain_old(T)` and convert back to a code word, evaluated at the four support points:

Support point	`T` code	`Gain_CW_old(T)`	`Gain_CW_target(T)`
−40 °C	368	439	447.909
`TCAL` = 25 °C	1408	342	350.000
100 °C	2608	228	234.932
150 °C	3408	166	172.351

`Gain_CW_target(TCAL)` is not recomputed — it is the measured result of Phase 2.

Step 5 — cubic fit (eq. 22–23): solve the 4×4 linear system $Ax=b$ for $x=[BASE_{new}, TL_{new}, TQ_{new}, TT_{new}]$ with the Vandermonde matrix of the four `T` codes and $b=Gain_CW_{target}(T)$. The solution is unique (Cramer's rule, LU decomposition or a least-squares helper such as Excel `LINEST`).

Step 6 — back to EEPROM coefficients (eq. 24):

$$g_base_{new}=BASE_{new}\quad g_tc_tl_{new}=TL_{new}\times 2^{11}\quad g_tc_tq_{new}=TQ_{new}\times 2^{22}\quad g_tc_tt_{new}=TT_{new}\times 2^{35}$$

Round each result to its integer bit width and range-check it against the field widths in the EEPROM field table before packing. A coefficient that overflows its field indicates a measurement problem — abort and restore the original image rather than truncating.

Phase 5 — Program and Verify

Pack the new coefficients into words 3, 4, 5 and 8 of the **image that was read in Phase 1**, leaving every other bit untouched, then recalculate the CRC.

```
/* eeprom[] holds the image read in Phase 1; the *_new values are the results
 * of Phase 4. Signed coefficients are masked to their field width. */
eeprom[3] = (uint16_t)(((uint16_t)(g_tc_tl_new & 0x0FFF) << 4)
                     |  (uint16_t)(g_base_new  & 0x000F));
eeprom[4] = (uint16_t)(((uint16_t)(g_tc_tq_new & 0x03FF) << 6)
                     | (uint16_t)((g_base_new >> 4) & 0x003F));
eeprom[5] = (uint16_t)(((uint16_t)(g_tc_tt_new & 0x07FF) << 5)
                     | (uint16_t)(((g_base_new >> 10) & 1u) << 4)
                     |  (uint16_t)(eeprom[5] & 0x000Fu));          /* keep s_base */
eeprom[8] = (uint16_t)((eeprom[8] & 0xFF00u)                       /* keep epk_base */
                     |  (uint16_t)(o_base_new & 0x00FFu));

eeprom[2] = (uint16_t)((eeprom[2] & 0xFF00u) | eepromCrcCalc(eeprom));
```

```
[host] BASE_new = 477.8766, TL_new = -7.647472e-02, TQ_new = -1.4641e-05, TT_new = 3.1618e-09
[host] g_base_new  =       478          -> word 3 bits 3:0, word 4 bits 5:0, word 5 bit 4
[host] g_tc_tl_new = 0xF63 = -157       -> word 3, bits 15:4
[host] g_tc_tq_new = 0x3C3 =  -61       -> word 4, bits 15:6
[host] g_tc_tt_new = 0x06D =  109       -> word 5, bits 15:5
[host] o_base_new  = 0x0B  =   11       -> word 8, bits 7:0

[host] word  3: 0xF5E6 -> 0xF63E        g_tc_tl and g_base bits 3:0
[host] word  4: 0xF31D -> 0xF0DD        g_tc_tq and g_base bits 9:4
[host] word  5: 0x0BE1 -> 0x0DA1        g_tc_tt and g_base bit 10 (s_base bits 3:0 preserved)
[host] word  8: 0x0007 -> 0x000B        o_base 7 -> 11 (epk_base bits 15:8 preserved)
[host] eepromCrcCalc() = 0x6C           recalculated over the modified image
[host] word  2: 0x00FE -> 0x006C        CRC patched

TX  AA 13 00 00 00                      cmdResetSensor (SICI was disabled by the sweeps)
RX  13 00
                                        ... wait 300 ms ...
TX  AA 11 00 00 00                      cmdEnterTestMode (re-enter SICI, ISM down)
RX  11 00

TX  AA 26 00 24 00                      cmdProgramEEPROM
RX  26 00                               intermediate ack
TX  C0 30 57 47 00 6C F6 3E             words 0..3
TX  F0 DD 0D A1 49 9D 45 85             words 4..7
TX  00 0B 00 F7 00 23 FE F2             words 8..11
TX  BC 30 F7 C2 44 CF 1C 70             words 12..15
TX  FE 65 DB 40                         words 16..17
RX  30 00                               final ack (CMD_ECHO = data[1] = 0x30)

                                        ... wait 500 ms for the EEPROM write ...

TX  AA 10 00 00 24                      cmdReadEEPROM (verify BEFORE resetting)
RX  10 24
RX  C0 30 57 47 00 6C F6 3E             ... 36 bytes ...

[host] pre-reset read-back: MATCH (18/18 words), CRC VALID

TX  AA 13 00 00 00                      cmdResetSensor (activate the new content)
RX  13 00
                                        ... wait 300 ms ...

TX  AA 11 00 00 00                      cmdEnterTestMode
RX  11 00
TX  AA 10 00 00 24                      cmdReadEEPROM (confirm after reset)
RX  10 24
RX  C0 30 57 47 00 6C F6 3E             ... 36 bytes ...

[host] post-reset read-back: MATCH (18/18 words)
[host] CRC stored = 0x6C, CRC calculated = 0x6C -> VALID
```

> **Verify before you reset.** Read the image back while the part is still in SICI test mode and only issue `cmdResetSensor` once the content and the CRC check out. A reset activates whatever is in the EEPROM — if the write was partial, the reset is what turns a recoverable situation into a device that boots with a CRC error and pulls both OCD outputs to GND. As long as you have not reset, the old content is still active and SICI is still available, so you can simply reprogram.

Finally, power-cycle the device once more and re-measure sensitivity and offset in normal operating mode (`cmdResetSensor` → wait → `cmdEnableMux` → `cmdStartReadout`). If both match the targets, the calibration is complete.

Rollback

If the verification measurement does not meet the targets, **restore the original image** before retrying:

```
TX  AA 26 00 24 00                    cmdProgramEEPROM with the Phase 1 backup
RX  26 00
TX  C0 30 57 47 00 FE F5 E6 ...       original 36 bytes, original CRC 0xFE
RX  30 00
TX  AA 13 00 00 00                    cmdResetSensor
RX  13 00
```

Then restart the procedure from Phase 1. The backup image already carries a valid CRC, so no recalculation is needed for the rollback.

DCW Sequence Summary

#	Phase	Action	Frame
1	1	Reset sensor	`AA 13 00 00 00`
2	1	Enter test mode	`AA 11 00 00 00`
3	1	**Read and archive the full EEPROM image**	`AA 10 00 00 24`
4	1	Read `Gain_CW` (`0x20`)	`AA 17 00 02 00` + `00 20`, `AA 18 00 00 02`
5	1	Read `Offset_CW` (`0x21`)	`AA 17 00 02 00` + `00 21`, `AA 18 00 00 02`
6	1	Read `TINT` (`0x18`) → `TCAL`	`AA 17 00 02 00` + `00 18`, `AA 18 00 00 02`
7	2	For `Gain_CW` = 210 and 570: reset → test mode → write `0x20` → write `0x25` = `0x9000` → mux → readout	see Phase 2
8	2	Interpolate `Gain_CW_target` (host side)	—
9	3	For `Offset_CW` = −40 and +40: reset → test mode → write `0x20` = target, write `0x21` → write `0x25` = `0x9000` → mux → readout	see Phase 3
10	3	Interpolate `Offset_CW_target` (host side)	—
11	4	Recalculate `o_base`, `g_base`, `g_tc_tl`, `g_tc_tq`, `g_tc_tt` (host side)	—
12	5	**Recalculate the CRC and patch word 2 (host side)**	—
13	5	Program the EEPROM	`AA 26 00 24 00` + 36 bytes
14	5	Reset, read back, verify image and CRC	`AA 13 00 00 00`, `AA 11 00 00 00`, `AA 10 00 00 24`
15	5	Reset, mux, readout — confirm sensitivity and offset	`AA 13 00 00 00`, `AA 29 00 02 00`, `AA 14 00 00 00`
16	—	On failure: reprogram the Phase 1 backup and repeat	`AA 26 00 24 00` + backup

Pitfalls

Symptom	Cause
Measurement is identical for both code words	The ISM was reactivated (a `cmdResetSensor` between the register write and the measurement) and overwrote the code word
AOUT reads a static level, no signal	`0x25` was left at `0x8000` — SICI still owns the AOUT pin. Write `0x9000`
A register write is rejected after the first measurement	SICI is disabled until the next chip reset. Power-cycle before writing again
Offset moves in the wrong direction	`Offset_CW` written as two's complement instead of sign-magnitude. −40 must be sent as `0xA8`, not `0xD8`
OCD outputs pulled to GND after the calibration	The CRC was not recalculated after modifying the coefficient words
Sensitivity matches but drifts over temperature	Only the room-temperature point was fitted; verify that all four support points entered the cubic fit